Lecture01: Introduction to Javascript

Why JavaScript Exists

The story of Netscape, Sun, and Microsoft and a language built in ten days

Before you write a single line of JavaScript, it helps to know *why it exists at all*. Because here is a fair question: the world already had C++, Java, and Python. Powerful, proven languages. So why invent yet another one? The answer is a genuine story — a corporate war, a ten-day scramble, and a "silly little brother" language that outlived every rival. This is that story.

1. The Web Was Born Dead

In the early 1990s, a web page was a static document. It could show text and images, and that was essentially all. There was no way for a page to react to you. If you filled in a form and made a mistake, the page could not tell you on the spot. It had to ship your data all the way to a distant server and wait for the server to send back a reply just to say "that's wrong, try again."

Netscape makers of the dominant browser of the era realised the web needed to come alive. It needed to respond instantly, right there in the page. That need is the seed of everything that follows.

2. Enter Sun, and a Language Called Java

At the same time, a company called **Sun Microsystems** had created a programming language named **Java**. Sun's main business was selling powerful, expensive computers that ran the internet's infrastructure — one writer called Sun the internet's primary "arms dealer."

Java's superpower was a promise: **"write once, run anywhere."** A Java program didn't care whether you were on Windows, Mac, or Linux it would run the same everywhere, inside a piece of software called the Java Virtual Machine. For a web full of all kinds of computers, that sounded perfect.

So in 1995, Sun and Netscape formed a partnership. Netscape would put Java into its browser. The two giants of the early web the biggest browser and the biggest internet-server company joined forces.

3. The Secret War Against Microsoft

But there was a deeper motive behind the alliance, and it's the heart of the drama. Both Sun and Netscape had a common enemy: Microsoft.

In the 1990s, Microsoft's power came from one fact everyone needed Windows to run software. Programs were written for Windows specifically, so if you wanted to use them, you had to have Windows. Microsoft owned the ground that all software stood on.

Sun and Netscape had a dream to break that grip. What if all software ran *inside the browser* instead, written in Java? Then it wouldn't matter what operating system you had underneath. The browser would become the new ground, and Windows would become irrelevant reduced, as one history put it, to little more than a particularly expensive way to switch the computer on.

If software lived in the browser, who would pay extra for Windows? That was the threat Microsoft genuinely feared and it was right to.

4. Why a SECOND Language Was Needed

Here is the question students always ask: if Java was already going into the browser, **why invent JavaScript at all?** The answer is that Java, for all its power, was the wrong shape for one specific job.

Netscape pictured two very different kinds of work on a web page:

- **The heavy lifting** — big, self-contained programs sitting in a box on the page (a chart tool, a small game). This was Java's territory, written by professional programmers.
- **The glue** — small touches that tie the page together: react to a click, check a form field, change some text. This needed to be simple enough for web designers to write.

Java was badly suited to the glue job for three concrete reasons. It ran in a sealed box and couldn't easily touch the page around it. It was too heavyweight — firing up the whole Java engine just to check a phone number was like starting a truck to open a garage door. And it was too hard for the web designers who were the intended authors.

So Netscape decided it needed ***both***: Java as the powerful "component language," and a new, light "glue language" for everyone else. That glue language is JavaScript.

5. Ten Days in May

Netscape hired an engineer named **Brendan Eich**, originally to put an existing language (Scheme) in the browser. Instead, he was asked to build the new glue language — and the prototype was written in roughly **ten days** in May 1995.

There was one strange constraint, and it's the most interesting twist of all. Because of the Sun partnership, marketing demanded the new language *look like Java* to ride its popularity. Eich described being under orders to make it Java's "silly little brother." That single business decision "make it look like Java" is what ruled out simply adopting Python or Perl, which existed and would have fit. The best technical choice lost to a marketing requirement.

The name itself was pure marketing: it was called **JavaScript** to borrow Java's fame, even though the two languages are largely unrelated. As the saying goes: Java and JavaScript are as related as "car" and "carpet."

6. Two Languages, Side by Side

For years, the two languages lived together in the browser, each doing its own job:

Java (the “component” language)	JavaScript (the “glue” language)
Powerful, fast, full-featured	Light, simple, forgiving
Written by professional programmers	Written by web designers
Ran in a sealed box (applet)	Woven into the page itself
Could NOT easily touch the page	Could touch any button, text, element
Needed a plugin + slow startup	Built in, ran instantly
Removed from browsers (~2015–2017)	Became the language of the web

7. How the Little Brother Won

Java was the more powerful language. So how did JavaScript take over the frontend entirely? Not by being better by being in the right place.

- **No plugin, no waiting.** Java needed a plugin installed, and the engine booted slowly on every page. JavaScript was already inside every browser and ran instantly.
- **Security.** The Java plugin became a notorious source of security holes, and browser makers fenced it off harder and harder.
- **JavaScript caught up.** Around 2011 onward, browsers gained fast JavaScript engines and native graphics erasing Java’s speed and visual advantages.

- **It lived in the page.** Java sat in a box; JavaScript was woven into the document. As the web became about fluid, integrated pages, that was exactly the right shape.

Browsers dropped support for Java applets entirely around 2015–2017. The powerful component language was evicted from the browser. The silly little glue language became the foundation of the modern web.

8. The Same Story, Today

Here is the beautiful part. A modern technology called **WebAssembly** now lets powerful languages like C++ and Rust run in the browser at near-native speed. Sounds like Java all over again and yet, for all its power, WebAssembly ***cannot touch the page***. To change a button or react to a click, it must call out to JavaScript.

The exact word used in 1995 to describe JavaScript's role *"glue"* is the exact word used today to describe its role next to WebAssembly. Thirty years on, the heavyweight component keeps changing (Java, then C++ via WebAssembly), but the glue never does.

Microsoft feared the right outcome but got the threat's name wrong. The dream of OS-independent software in the browser largely came true delivered not by mighty Java, but by the little language nobody took seriously.

The One-Sentence Takeaway

Those other languages own the computer. JavaScript owns the browser tab and the browser turned out to be a very big room. It didn't win by being the most powerful. It won by being woven into the page, already there, ready to run. **And that is the language you start learning today.**

Developer know html and CSS, why do we need javascript

1: We can't put C++ in the browser , it's too heavy, unsafe, and inaccessible. Our users are not kernel developers; they're web authors who just learned `<table>` and `<font>`. We need something lightweight, interpreted, forgiving, and safe.

```
#include<iostream> using namespace std; int main() { cout << "Hello World"; }
```

```
console.log("Hello World")
```

2: Massive Security Nightmare:

- **Unrestricted Access:** C++ gives you low-level control over memory and system calls. If a browser ran arbitrary C++ code from a website, that code could easily:
  - Read/write any file on your computer.
  - Install malware.
  - Access your webcam or microphone without permission.
  - Crash your entire operating system.

1. File system access

```
#include <fstream> std::ofstream file("C:\\Users\\rohit\\secrets.txt"); file << "st
olen data";
```

- Without sandboxing, this code could read/write/delete any file on your machine.
- In a sandboxed environment, you'd have to **intercept all file I/O** calls and either block them or restrict them to a safe "virtual" file system.

---

2. System calls (executing programs)

```
#include <cstdlib> system("rm -rf /"); // Linux system("format C:"); // Windows 95 nightmare
```

- Raw C++ can call `system()` to run OS commands.
- Sandboxing would mean completely **disabling or trapping** such calls, otherwise a website could literally wipe your drive.

---

3. Direct memory access (pointers)

```
int* p = (int*)0xB8000; // Access video memory *p = 42;
```

- C++ allows arbitrary pointer arithmetic → could overwrite OS/kernel memory or peek into sensitive regions.
- In a sandbox, you'd have to **rewrite the runtime** so pointers never escape into raw machine addresses.

---

4. Networking

```
#include <sys/socket.h> connect(...); // Open a raw socket to exfiltrate data
```

- C++ can open arbitrary sockets, bypassing the browser's control.
- Sandboxing would require blocking direct socket creation and only allowing **browser-controlled HTTP requests**.

3: System Configurate was very less like 4-8mb ram, 200-400mb hard disk.

Typical Home PC Specs in 1995

- **RAM:**
  - Average consumer PCs had **4 MB to 8 MB** of RAM.
  - Higher-end machines (for developers/enthusiasts) sometimes had **16 MB**.
  - Anything beyond that was rare and expensive.
- **Hard Disk:**
  - Common sizes: **200 MB – 500 MB**.
  - Higher-end PCs: **1 GB** drives were just starting to appear.
  - Compare that to today's **1 TB SSDs** 😅.

- **CPU:**
  - Intel **Pentium 75–133 MHz** was mainstream.
  - 486 processors were still common in cheaper systems.

🔹 Why this mattered for C++ vs JS in browsers

- Running a **sandboxed C++ runtime** would've eaten up tons of RAM and CPU → impossible when you only had 8 MB of RAM total, shared with Windows 95 and the browser itself.
- Hard disks were small and slow → no space for large runtime environments or heavy libraries.
- Browsers had to stay **lightweight** or else people simply wouldn't use them.

4: Automatic Memory Management (Garbage Collection):